

SHETH L.U.J & SIR M.V. COLLEGE
PRACTICAL NO2

Aim: Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

1. CODE:

```
from math import gcd

# Check if number is prime
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False
    return True

# Extended Euclidean Algorithm
def extended_gcd(a, b):
    if b == 0:
        return (1, 0, a)
    x1, y1, g = extended_gcd(b, a % b)
    x = y1
    y = x1 - (a // b) * y1
    return (x, y, g)

# Modular Inverse
def mod_inverse(e, phi):
    x, y, g = extended_gcd(e, phi)
    if g != 1:
        return None
    return x % phi

# ----- Main Program -----

p = int(input("Enter prime number p: "))
q = int(input("Enter prime number q: "))

if not is_prime(p) or not is_prime(q):
    print("Error: Both numbers must be prime.")
    exit()

NAME:SIDDHANT CHALKE
ROLLNO:T073
SUBJECT:CYBER SECURITY
```

SHETH L.U.J & SIR M.V. COLLEGE
PRACTICAL NO2

```
if p == q:
    print("Error: p and q should be different.")
    exit()

n = p * q
phi = (p - 1) * (q - 1)

print("\nValue of n =", n)
print("Value of phi(n) =", phi)

e = int(input("Enter public key e: "))

while e <= 1 or e >= phi or gcd(e, phi) != 1:
    print("Invalid e! Enter another value.")
    e = int(input("Enter public key e: "))

d = mod_inverse(e, phi)

if d is None:
    print("Error: Modular inverse does not exist.")
    exit()

print("\nPublic Key :", (e, n))
print("Private Key:", (d, n))

message = input("\nEnter message: ")

# Ensure n is large enough
for ch in message:
    if ord(ch) >= n:
        print("\nError: n is too small for this message.")
        print("Choose larger prime numbers.")
        exit()

# Encryption
encrypted = [pow(ord(ch), e, n) for ch in message]

print("\nEncrypted Message:")
print(encrypted)

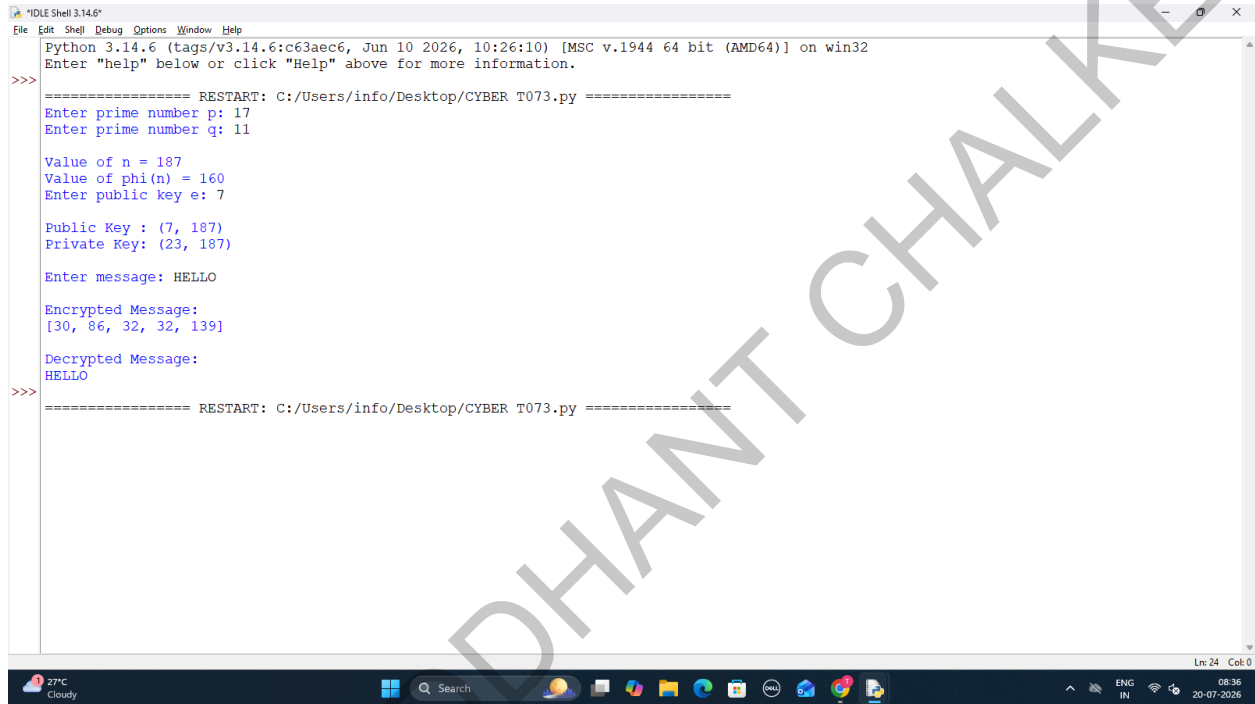
NAME:SIDDHANT CHALKE
ROLLNO:T073
SUBJECT:CYBER SECURITY
```

SHETH L.U.J & SIR M.V. COLLEGE
PRACTICAL NO2

```
# Decryption
decrypted = ''.join(chr(pow(c, d, n)) for c in encrypted)

print("\nDecrypted Message:")
print(decrypted)
```

OUTPUT:



```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/info/Desktop/CYBER T073.py =====
Enter prime number p: 17
Enter prime number q: 11

Value of n = 187
Value of phi(n) = 160
Enter public key e: 7

Public Key : (7, 187)
Private Key: (23, 187)

Enter message: HELLO

Encrypted Message:
[30, 86, 32, 32, 139]

Decrypted Message:
HELLO
>>>
===== RESTART: C:/Users/info/Desktop/CYBER T073.py =====
```

GUI:

```
import tkinter as tk
from tkinter import messagebox
from math import gcd
```

```
# Check if number is prime
def is_prime(num):
    if num < 2:
        return False
    for i in range(2, int(num ** 0.5) + 1):
        if num % i == 0:
            return False
    return True
```

NAME:SIDDHANT CHALKE
ROLLNO:T073
SUBJECT:CYBER SECURITY

SHETH L.U.J & SIR M.V. COLLEGE
PRACTICAL NO2

```
# Find modular inverse
def mod_inverse(e, phi):
    for d in range(2, phi):
        if (d * e) % phi == 1:
            return d
    return None
```

```
cipher = []
private_d = None
private_n = None
```

```
# Encrypt function
```

```
def encrypt():
    global cipher, private_d, private_n

    try:
        p = int(entry_p.get())
        q = int(entry_q.get())
        e = int(entry_e.get())
        message = entry_msg.get()

        if not is_prime(p) or not is_prime(q):
            messagebox.showerror("Error", "p and q must be prime numbers.")
            return

        if p == q:
            messagebox.showerror("Error", "p and q should be different.")
            return

        n = p * q
        phi = (p - 1) * (q - 1)

        if e <= 1 or e >= phi or gcd(e, phi) != 1:
            messagebox.showerror("Error", "Invalid public key e.")
            return

        d = mod_inverse(e, phi)

        if d is None:
            messagebox.showerror("Error", "Private key could not be generated.")
```

NAME:SIDDHANT CHALKE
ROLLNO:T073
SUBJECT:CYBER SECURITY

SHETH L.U.J & SIR M.V. COLLEGE
PRACTICAL NO2

```
    return

for ch in message:
    if ord(ch) >= n:
        messagebox.showerror(
            "Error",
            "n is too small.\nChoose larger prime numbers."
        )
    return

cipher = [pow(ord(ch), e, n) for ch in message]

private_d = d
private_n = n

lbl_public.config(text=f"Public Key : ({e}, {n})")
lbl_private.config(text=f"Private Key : ({d}, {n})")
lbl_encrypt.config(text=str(cipher))
lbl_decrypt.config(text="")

except ValueError:
    messagebox.showerror("Error", "Enter valid numeric values.")

# Decrypt function
def decrypt():
    if not cipher:
        messagebox.showerror("Error", "Encrypt a message first.")
    return

    decrypted = "".join(chr(pow(c, private_d, private_n)) for c in cipher)
    lbl_decrypt.config(text=decrypted)

# GUI Window
root = tk.Tk()
root.title("RSA Encryption and Decryption")
root.geometry("650x550")
root.resizable(False, False)

tk.Label(root, text="RSA Encryption & Decryption",
        font=("Arial", 18, "bold")).pack(pady=10)
```

NAME: SIDDHANT CHALKE
ROLLNO: T073
SUBJECT: CYBER SECURITY

SHETH L.U.J & SIR M.V. COLLEGE
PRACTICAL NO2

```
tk.Label(root, text="Prime Number (p)").pack()  
entry_p = tk.Entry(root, width=30)  
entry_p.pack()
```

```
tk.Label(root, text="Prime Number (q)").pack()  
entry_q = tk.Entry(root, width=30)  
entry_q.pack()
```

```
tk.Label(root, text="Public Key (e)").pack()  
entry_e = tk.Entry(root, width=30)  
entry_e.pack()
```

```
tk.Label(root, text="Message").pack()  
entry_msg = tk.Entry(root, width=40)  
entry_msg.pack(pady=5)
```

```
tk.Button(root, text="Encrypt",  
          width=20,  
          bg="lightgreen",  
          command=encrypt).pack(pady=10)
```

```
lbl_public = tk.Label(root, text="", font=("Arial", 10))  
lbl_public.pack()
```

```
lbl_private = tk.Label(root, text="", font=("Arial", 10))  
lbl_private.pack()
```

```
tk.Label(root, text="Encrypted Message",  
          font=("Arial", 11, "bold")).pack(pady=5)
```

```
lbl_encrypt = tk.Label(root, text="", fg="blue", wraplength=600)  
lbl_encrypt.pack()
```

```
tk.Button(root, text="Decrypt",  
          width=20,  
          bg="lightblue",  
          command=decrypt).pack(pady=10)
```

```
tk.Label(root, text="Decrypted Message",
```

NAME: SIDDHANT CHALKE
ROLLNO: T073
SUBJECT: CYBER SECURITY

SHETH L.U.J & SIR M.V. COLLEGE
PRACTICAL NO2

```
font=("Arial", 11, "bold")).pack()
```

```
lbl_decrypt = tk.Label(root, text="", fg="red", font=("Arial", 12))  
lbl_decrypt.pack()
```

```
root.mainloop()
```

OUTPUT:

RSA Encryption and Decryption

RSA Encryption & Decryption

Prime Number (p)
61

Prime Number (q)
53

Public Key (e)
17

Message
HELLO

Encrypt

Public Key : (17, 3233)
Private Key : (2753, 3233)

Encrypted Message
[3000, 28, 2726, 2726, 1307]

Decrypt

Decrypted Message
HELLO

NAME:SIDDHANT CHALKE
ROLLNO:T073
SUBJECT:CYBER SECURITY